CS-781 Capstone, Phase 5: Legal Statute Identification

A multi-label chunk classifier on the four Phase-1 language models

Manish Kumar (251330604)
CS-781, IIT Kanpur

October 2026

Abstract

We assign the relevant statutes (100 statute ids, several per document) to Indian legal documents, starting from the four legal language models pre-trained in Phase 1. A document is cut into 512-token chunks; eight chunks are encoded by the Phase-1 backbone, read by a BiGRU and pooled by one attention query per statute. Ten first-round experiments cover all four architectures and eight adaptation techniques (full and partial fine-tuning, LoRMA, QLoRA, AdaLoRA, Houlsby adapters, prefix and prompt tuning) plus knowledge distillation; later rounds train longer and distil from growing ensembles of earlier runs. All scores are macro-F1 on a cleaned validation split with per-statute thresholds *cross-fitted* (fitted on one half, scored on the other). The best single Phase-1 model is a 6-block A4 student distilled from an 8-model teacher (0.411); an MLP stacker over 27 Phase-1 runs reaches 0.423, and a hybrid stacker that adds lexical TF-IDF models reaches 0.460. All numbers are generated from the run outputs by `p5/report/make_figures.py`.

Contents

1 Introduction **2**

2 Data and integrity checks **2**

3 Methods **3**
3.1 Hierarchical multi-label model . . . 3
3.2 Adaptation techniques . . . 3
3.3 Training, selection and thresholds . . . 4
3.4 Ensembles and stacking . . . 4
3.5 Submissions . . . 4

4 Results **5**
4.1 All experiments . . . 5
4.2 Thresholds . . . 6
4.3 Rare and frequent statutes . . . 6
4.4 Missed and confused statutes . . . 7
4.5 Knowledge distillation and efficiency . . . 8
4.6 Ensembles, stacking and lexical models . . . 8
4.7 Submission files . . . 9

5 Analysis **10**

6 Limitations and next steps **10**

7 Conclusions **11**

8 Reproducibility **11**

1 Introduction

Legal statute identification (LSI) asks which of 100 statutes apply to a case document; a document can cite several. The primary metric is macro-F1 over statutes, so rare statutes weigh as much as frequent ones; micro-F1 and subset accuracy are secondary. In this corpus the statute names are masked in the text (`<SECTION>`, `<ACT>`), so the model must infer statutes from the facts, not read them off. The rules require the Phase-1 models as the only pretrained weights, all four architectures, at least six substantially different experiments with at least five adaptation techniques, LoRMA and knowledge distillation.

2 Data and integrity checks

Table 1: Splits used (lengths in Phase-1 BPE tokens).

Split	Documents	Statutes / doc	Median length	90th pct.	Tokens
train	42,315	3.71	646	4,171	72.3M
val	10,024	3.91	822	4,513	19.3M
test	13,019	–	796	4,285	24.1M

Cleaning (`p5/lsi/data.py`).

- `train.json` has 42,750 entries; 435 are exact duplicates of another entry's text (whitespace-normalised hash). Duplicates are merged and their label sets unioned (80 duplicated texts carry different label sets), leaving 42,315 training documents.
- `dev.json` (10,181 documents) contains 96 texts that also occur in the test file and 118 that occur in training. They are removed, leaving the 10,024 validation documents used for every selection decision.

Statute frequencies are long-tailed: the most frequent statute occurs in 11,577 training documents, the rarest in 270, and the ten most frequent account for 41% of all labels (Figure 1).

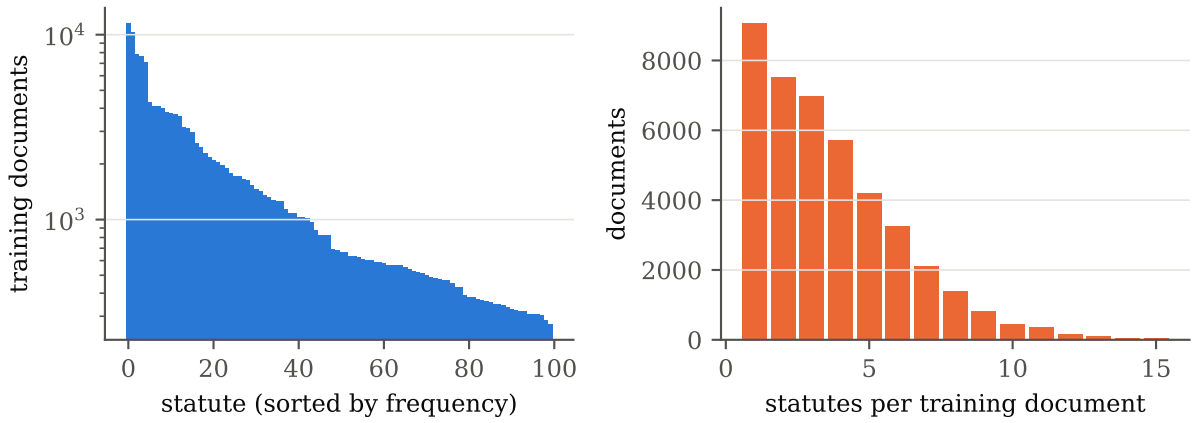


Figure 1: Left: training documents per statute. Right: statutes per training document.

3 Methods

3.1 Hierarchical multi-label model

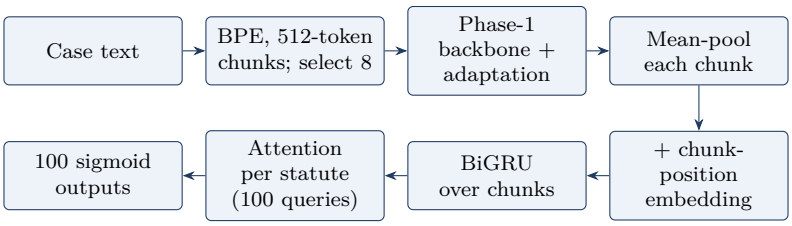


Figure 2: The LSI model (`p5/lsi/model.py`).

- **Chunking** (`data.strategy`). Eight chunks of 512 tokens: *spread* evenly over the document (default; the whole document when it is at most eight chunks long, 88% of validation documents), or *evidence* (V1): chunks centred on the masked `<SECTION>`/`<ACT>` placeholders.
- **Document layer**: projection to 384 dimensions, a learned chunk-position embedding, a BiGRU over the chunks and *per-statute attention*: each statute has its own query vector over the chunk states (a statute is often discussed in one part of the judgment) and its own output vector.
- **Loss**: binary cross-entropy per statute with positive weights $\sqrt{n_{\text{neg}}/n_{\text{pos}}}$ (capped at 30) to give rare statutes more weight.
- **Backbones**: as in Phases 3–4, A1 and A4 keep their causal mask, A3 reads each chunk as a bidirectional prefix and A2 uses its decoder states; all start from the cleaned 1B-token Phase-1 checkpoints.

3.2 Adaptation techniques

- **Full fine-tuning** (E2 A1; later W2/X2 A4, Y4 A3, Y5 A2).
- **Partial fine-tuning** (E4 A2, E5 A4): the top four blocks and the final norm.
- **LoRMA** (E1, W4; A4) [1]: multiplicative low-rank update $W = (I + \frac{\alpha}{r}BA)W_0$, $r = 16$, $\alpha = 32$, on attention and MLP projections.
- **QLoRA** (E6, W5, X1; A1) [2]: the frozen backbone stored in 4-bit NF4 (block 64), LoRA $r = 16$ on top.
- **AdaLoRA** (E7; A3) [7]: SVD-form update, rank pruned from 12 to 8.
- **Houlsby adapters** (E3, W3; A3) [4]: bottleneck 64.
- **Prefix tuning** (E8; A4) [6]: 16 learned key/value positions per layer from an MLP.
- **Prompt tuning** (E9; A1) [5]: 20 learned input embeddings.
- **Knowledge distillation** [3] (E10, W6, X3, Y1, Y2, Y3): the student is A4 with 6 of its 10 blocks (0, 2, 4, 5, 6, 8; all global-attention layers kept) or A1 with 6 of 8 (Y3), fully fine-tuned with $\alpha\,\text{BCE} + (1-\alpha)\,T^2\,\text{BCE}(\sigma(z_s/T), \sigma(z_t/T))$, $T = 2$. The teacher is the average logit of an ensemble of earlier runs, computed once on the training set (`dump_logits.py`): TEACHER = E2, E3, E5, TEACHER2 = E1, E3, E6, V2, V3, TEACHER3 = W1, W2, W3, W5, W6, TEACHER4 = W1, W2, W3, W4, W5, W6, X1, X3. Y2 uses $\alpha = 0.3$, the others 0.5.

3.3 Training, selection and thresholds

AdamW, 16 documents per step, 5% warm-up then cosine decay, weight decay 0.01, gradient clipping 1.0, bf16. First round: 3 epochs; learning rate 5×10^{-5} for full fine-tuning and KD students, 10^{-4} for partial fine-tuning, 2×10^{-4} for LoRMA, QLoRA, AdaLoRA and adapters, 10^{-3} for prefix and 3×10^{-3} for prompt tuning; the document layer always uses 10^{-3}. Later rounds train 5–8 epochs. Every 800–2,000 steps a fixed 2,000-document subset of validation is scored and the best state is kept.

Thresholds. A statute is predicted when its probability exceeds its threshold. Per-statute thresholds are chosen on validation (grid search, at least 5 validation positives, else the global threshold) and every document gets at least one statute. Because fitting 100 thresholds on the same data inflates the score, every number in this report is *cross-fitted*: thresholds fitted on one half of validation score the other half and vice versa (`make_submission.py`). The test submission uses thresholds fitted on all of validation.

3.4 Ensembles and stacking

- **Mean ensembles** (ENS*): average probability of several runs, members picked greedily on validation (`ensemble_select.py`).
- **MLP stacker** (`stack.py`): input = the logits of every run for all 100 statutes, one hidden layer of 512, dropout 0.3, 30 epochs; trained 5-fold on the validation split, so every validation document is scored by a stacker that did not see it; the 5 fold models (and several seeds) are averaged for test.
- **Per-statute GBM stacker** (`stack_gbm.py`): for statute j, LightGBM over every run's logit for j, the mean logit of all statutes and the across-run spread of the logit for j; same 5-fold scheme. STACK9–11 average the GBM and MLP stackers.
- **Lexical models** (T*, M1): TF-IDF features with one-vs-rest logistic regression or a linear SVM (`tfidf_lr.py`), trained on the training split only (T6: on training plus the other validation folds). They use no pretrained weights; stacks that include them are reported as *hybrid*.

3.5 Submissions

`make_submission.py` writes `prediction.json`, a list of `{"id", "pred"}` with the predicted statute ids for every test document in order, at the root of a zip.

4 Results

4.1 All experiments

Table 2: Validation results (10,024 documents). Global = macro-F1 at one tuned global threshold (in-sample); CF = cross-fitted per-statute thresholds (honest); P, R, micro-F1 and subset accuracy at CF. Above the line: first round (3 epochs); below: longer training and distillation rounds.

Run	Arch	Method	Ep.	Trainable	%	Global	CF	P	R	Micro	Subset
E1	A4	LoRMA	3	3.27M	3.6	0.3692	0.3771	0.356	0.421	0.477	0.039
E2	A1	full fine-tuning	3	83.13M	100.0	0.3501	0.3606	0.337	0.412	0.462	0.035
E3	A3	Houlsby adapters	3	2.67M	3.2	0.3569	0.3642	0.342	0.412	0.464	0.033
E4	A2	partial (top 4 blocks)	3	27.29M	30.2	0.3301	0.3391	0.308	0.404	0.439	0.025
E5	A4	partial (top 4 blocks)	3	26.24M	29.6	0.3582	0.3700	0.349	0.423	0.467	0.040
E6	A1	QLoRA (NF4)	3	2.83M	10.0	0.3619	0.3705	0.350	0.420	0.468	0.034
E7	A3	AdaLoRA	3	2.39M	2.9	0.3403	0.3533	0.335	0.402	0.455	0.029
E8	A4	prefix tuning	3	4.10M	4.5	0.3474	0.3526	0.327	0.404	0.452	0.030
E9	A1	prompt tuning	3	1.08M	1.3	0.3372	0.3453	0.320	0.399	0.448	0.031
E10	A4	KD student (6 blocks, TEACHER)	3	63.40M	100.0	0.3744	0.3816	0.362	0.425	0.479	0.040
V1	A4	partial (top 4 blocks)	3	26.24M	29.6	0.3601	0.3705	0.350	0.422	0.464	0.033
V2	A4	partial (top 4 blocks)	5	26.24M	29.6	0.3769	0.3813	0.364	0.422	0.476	0.043
V3	A1	full fine-tuning	5	83.13M	100.0	0.3673	0.3715	0.355	0.414	0.471	0.043
W1	A4	partial (top 4 blocks)	8	26.24M	29.6	0.3822	0.3884	0.382	0.413	0.489	0.053
W2	A4	full fine-tuning	5	88.57M	100.0	0.3768	0.3853	0.375	0.418	0.486	0.047
W3	A3	Houlsby adapters	5	2.67M	3.2	0.3693	0.3763	0.357	0.417	0.477	0.039
W4	A4	LoRMA	5	3.27M	3.6	0.3842	0.3882	0.371	0.425	0.487	0.044
W5	A1	QLoRA (NF4)	5	2.83M	10.0	0.3815	0.3879	0.372	0.429	0.482	0.041
W6	A4	KD student (6 blocks, TEACHER2)	6	63.40M	100.0	0.3931	0.3971	0.383	0.432	0.491	0.050
X1	A1	QLoRA (NF4)	8	2.83M	10.0	0.3833	0.3868	0.374	0.419	0.485	0.043
X2	A4	full fine-tuning	8	88.57M	100.0	0.3810	0.3813	0.374	0.405	0.483	0.049
X3	A4	KD student (6 blocks, TEACHER3)	8	63.40M	100.0	0.4007	0.4037	0.395	0.428	0.502	0.056
Y1	A4	KD student (6 blocks, TEACHER4)	8	63.40M	100.0	0.4009	0.4018	0.400	0.421	0.503	0.055
Y2	A4	KD student (6 blocks, TEACHER4)	8	63.40M	100.0	0.4076	0.4107	0.402	0.440	0.508	0.054
Y3	A1	KD student (6 blocks, TEACHER4)	8	68.95M	100.0	0.3963	0.3974	0.384	0.429	0.498	0.049
Y4	A3	full fine-tuning	5	82.34M	100.0	0.3535	0.3617	0.338	0.418	0.454	0.033
Y5	A2	full fine-tuning	5	90.43M	100.0	0.3473	0.3524	0.321	0.415	0.450	0.031

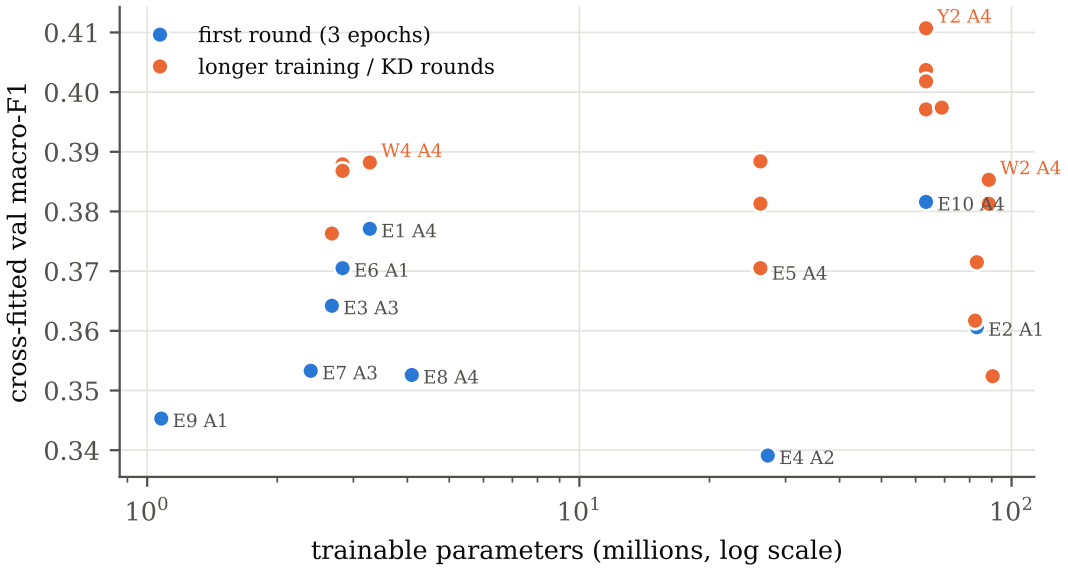


Figure 3: Cross-fitted macro-F1 against the number of trained parameters (document layer included).

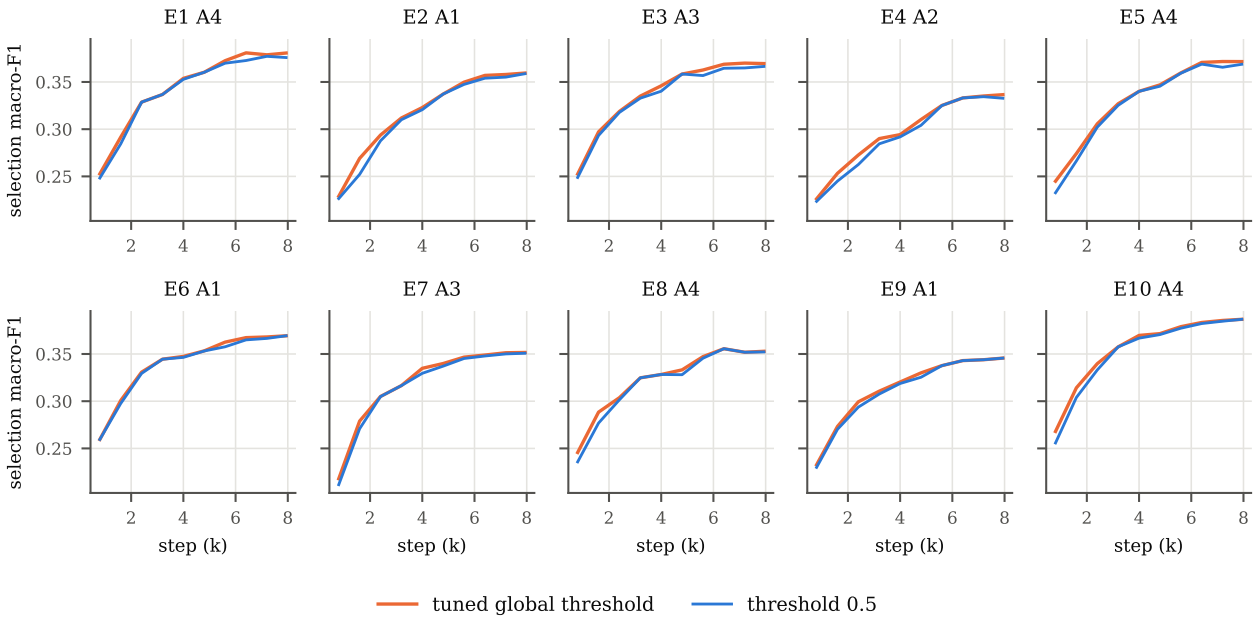


Figure 4: First-round runs: macro-F1 on the 2,000-document selection subset during training.

4.2 Thresholds

Table 3: Threshold strategies on the best single model (Y2).

Strategy	Macro-F1
0.5 for every statute	0.4076
one global threshold (0.55)	0.4080
per-statute thresholds, fitted on all of val (in-sample)	0.4266
per-statute thresholds, 2-fold cross-fitted (honest)	0.4107

4.3 Rare and frequent statutes

Table 4: Y2 per-statute F1 by training-frequency quintile (20 statutes each). Correlation of per-statute F1 with log frequency: 0.53.

Quintile	Training docs	Statutes	Mean F1	Min	Max
Q1	270–390	20	0.321	0.115	0.629
Q2	390–583	20	0.396	0.213	0.812
Q3	583–1,050	20	0.361	0.147	0.844
Q4	1,050–2,108	20	0.502	0.188	0.746
Q5	2,108–11,577	20	0.559	0.408	0.836

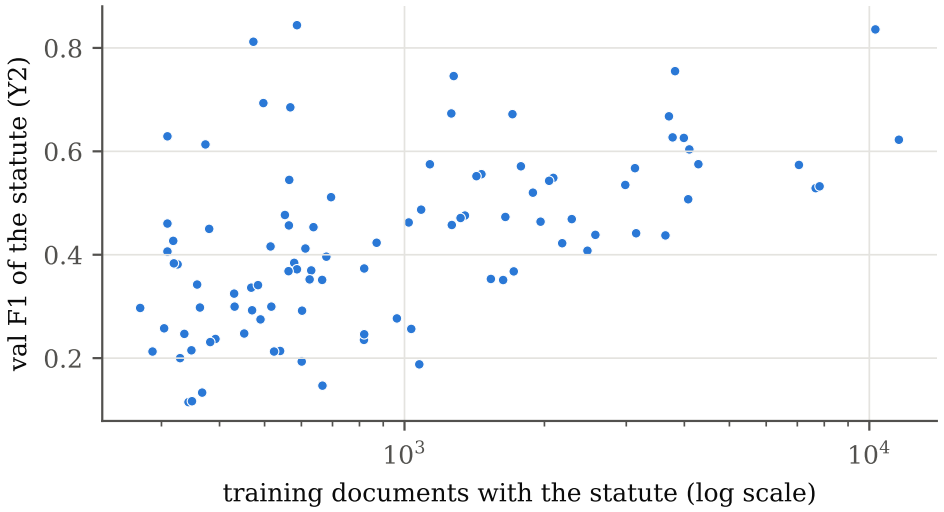


Figure 5: Per-statute validation F1 of Y2 against training frequency.

4.4 Missed and confused statutes

Table 5: Y2: frequent statutes (at least the median frequency) with the lowest validation F1.

Statute	Train docs	Val docs	F1	TP	FP	FN
52	1,076	271	0.188	61	317	210
9	818	206	0.235	64	274	142
41	819	201	0.246	64	255	137
1	1,032	255	0.256	73	242	182
7	1,034	258	0.257	69	211	189
83	963	242	0.277	120	505	122
63	1,630	404	0.351	188	479	216
85	1,534	386	0.353	196	528	190

Table 6: Y2: most frequent (missed gold statute, wrongly predicted statute) pairs in the same validation document, with how often the two statutes co-occur in training.

Missed	Predicted	Docs	Train (missed)	Train (predicted)	Co-occur in train
12	5	170	4,288	11,577	790
13	5	153	4,100	11,577	803
14	5	131	3,774	11,577	979
39	5	118	7,056	11,577	2,797
97	5	116	7,817	11,577	2,405
10	5	110	4,077	11,577	1,088
44	5	106	3,642	11,577	1,369
43	5	103	7,665	11,577	3,244
54	97	101	2,184	7,817	930
39	97	99	7,056	7,817	1,611

4.5 Knowledge distillation and efficiency

Table 7: Size, compute and memory. Test inference = seconds to score the 13,019 test documents.

Run	Arch	Method	Total	Trainable	Train (min)	Test infer. (s)	Peak GB	Ckpt MB
E1	A4	LoRMA	90.8M	3.27M	124	261	8.3	13.1
E2	A1	full fine-tuning	83.1M	83.13M	54	97	5.9	332.6
E3	A3	Houlsby adapters	83.9M	2.67M	57	132	3.8	10.7
E4	A2	partial (top 4 blocks)	90.4M	27.29M	60	207	4.0	109.2
E5	A4	partial (top 4 blocks)	88.6M	26.24M	55	141	3.9	105.0
E6	A1	QLoRA (NF4)	28.3M	2.83M	99	141	5.7	11.4
E7	A3	AdaLoRA	83.7M	2.39M	134	35	6.0	9.6
E8	A4	prefix tuning	91.6M	4.10M	99	112	4.9	16.4
E9	A1	prompt tuning	83.1M	1.08M	68	144	3.3	4.3
E10	A4	KD student (6 blocks, TEACHER)	63.4M	63.40M	71	55	5.8	253.6
W4	A4	LoRMA	90.8M	3.27M	196	181	8.3	13.1
W2	A4	full fine-tuning	88.6M	88.57M	154	92	9.2	354.3
Y2	A4	KD student (6 blocks, TEACHER4)	63.4M	63.40M	177	104	5.8	253.6
Y3	A1	KD student (6 blocks, TEACHER4)	69.0M	68.95M	148	100	4.6	275.8

The distilled students have 63.4M parameters against 88.6M for full A4. The KD chain is E10 (0.382) → W6 (0.397) → X3 (0.404) → Y1 (0.402) and Y2 (0.411), each distilled from a larger teacher ensemble; the best non-distilled single run is W4 LoRMA (0.388).

4.6 Ensembles, stacking and lexical models

Table 8: Lexical models (no pretrained weights; inputs to the hybrid stacks), cross-fitted.

Run	Model	Macro-F1	Micro-F1	Train (min)
T1_tfidf_word	tfidf_lr, word (1, 2)	0.4069	0.500	10.5
T2_tfidf_char	tfidf_lr, char_wb (3, 4)	0.3931	0.483	31.8
T3_tfidf_svm	linear SVM, word (1, 2)	0.4047	0.498	5.6
T5_tfidf_lr13	tfidf_lr, word (1, 3)	0.4063	0.500	15.9
T6_tfidf_cv	tfidf_lr on train + val folds	0.4305	0.524	43.1
M1_lexmlp	lexical MLP over TF-IDF	0.4049	0.504	6.1

Table 9: Ensembles and stackers (cross-fitted / out-of-fold validation scores). The plain mean of the 27 Phase-1 runs scores 0.408.

Name	Combination	Inputs	Macro-F1	P	R	Micro-F1
ENS1_5models	mean of probabilities	5 runs (Phase-1 only)	0.3825	0.367	0.424	0.485
ENS2_greedy4	mean of probabilities	4 runs (Phase-1 only)	0.3998	0.386	0.437	0.494
ENS7_greedy4	mean of probabilities	4 runs (Phase-1 only)	0.4190	0.412	0.445	0.514
STACK4_all	MLP stacker (5-fold on val)	22 runs (Phase-1 only)	0.4223	0.417	0.444	0.524
STACK6_all	MLP stacker (5-fold on val)	27 runs (Phase-1 only)	0.4233	0.425	0.442	0.520
STACK7_hybrid	MLP stacker (5-fold on val)	28 runs (1 lexical)	0.4436	0.452	0.452	0.544
STACK8_hybrid	MLP stacker (5-fold on val)	31 runs (4 lexical)	0.4473	0.457	0.452	0.546
STACK9_gbm_mlp	per-statute GBM + MLP stacker	32 runs (5 lexical)	0.4595	0.475	0.462	0.554
STACK10_seedavg	MLP + GBM stackers, seed-averaged	all runs + lexical	0.4602	0.475	0.460	0.556
STACK11_full	MLP + GBM stackers, seed-averaged	all runs + lexical	0.4603	0.475	0.459	0.558

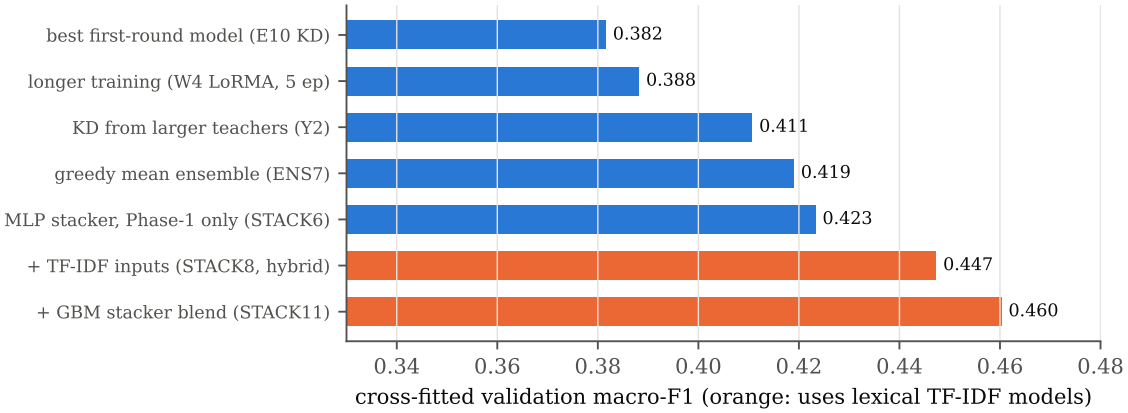


Figure 6: Steps from the best first-round model to the final submission.

4.7 Submission files

48 format-checked zips are in `p5/submissions/`. Uploaded first: `STACK11_full` (hybrid, 0.460), `STACK10_seedavg` (hybrid), `STACK6_all` (Phase-1 models only, 0.423), `Y2_kd4_a4_alpha03` (best single model, KD) and `W4_a4_lorma_ep5` (LoRMA).

To be written by the author — leaderboard scores

Add the Codabench test macro-F1, micro-F1 and subset accuracy of each uploaded submission and say which six or more count towards the requirement.

File
E10_kd_a4_small_20261027_035158.zip
E1_a4_lorma_20261027_031025.zip
E2_a1_full_20261027_023910.zip
E3_a3_adapter_20261027_023856.zip
E4_a2_partial_20261027_023925.zip
E5_a4_partial_20261027_023842.zip
E6_a1_qlora_20261027_033819.zip
E7_a3_adalora_20261027_041303.zip
E8_a4_prefix_20261027_034156.zip
E9_a1_prompt_20261027_031800.zip
ENS1_5models_20261027_023952.zip
ENS2_greedy4_20261027_035400.zip
ENS3_top7_20261027_035414.zip
ENS4_greedy5_20261027_121319.zip
ENS5_greedy5_20261027_122757.zip
ENS6_greedy3_20261027_224431.zip
ENS7_greedy4_20261028_042447.zip
ENS8_kd4_alldata_20261028_084240.zip
STACK10_seedavg_20261028_202312.zip
STACK11_full_20261028_212826.zip
STACK2_all_20261027_121446.zip
STACK3_all_20261027_122936.zip
STACK4_all_20261027_224405.zip
STACK6_all_20261028_042431.zip
STACK7_hybrid_20261028_182447.zip
STACK8_hybrid_20261028_192014.zip
STACK9_gbm_mlp_20261028_201838.zip
V1_e5_evidence_20261027_023938.zip
V2_e5_ep5_20261027_035321.zip
V2_e5_ep5_alldata_20261027_114221.zip
V3_e2_ep5_20261027_035101.zip
W1_a4_partial_ep8_20261027_121101.zip
W1_a4_partial_ep8_alldata_20261027_224144.zip
W2_a4_full_ep5_20261027_120601.zip
W3_a3_adapter_ep5_20261027_112243.zip
W4_a4_lorma_ep5_20261027_125109.zip
W5_a1_qlora_ep5_20261027_120257.zip
W5_a1_qlora_ep5_alldata_20261027_224128.zip
W6_kd2_a4_small_20261027_122530.zip
X1_a1_qlora_ep8_20261027_224040.zip

File
X2_a4_full_ep8_20261027_224100.zip
X3_kd3_a4_small_20261027_224113.zip
Y1_kd4_a4_20261028_041833.zip
Y2_kd4_a4_alpha03_20261028_041847.zip
Y2_kd4_alldata_20261028_084221.zip
Y3_kd4_a1_20261028_034904.zip
Y4_a3_full_ep5_20261028_021233.zip
Y5_a2_full_ep5_20261028_025608.zip

5 Analysis

To be written by the author — analysis of the results

The course rules do not allow LLM-written result analysis. Questions the course asks (§11.5), with the evidence above:

- Which adaptation techniques are effective for LSI, and can parameter-efficient methods retain full fine-tuning performance (Table 2, Figure 3)?
- How do long-document representation strategies affect statute identification (spread vs. evidence chunks: E5 vs. V1)?
- How do rare statutes affect performance (Table 4, Figure 5)?
- Which loss and class-balancing strategies are useful (positive weights, Section 3.3)?
- How do threshold strategies affect precision, recall and macro-F1 (Table 3)?
- Do multi-label relationships help (per-statute attention; the stackers' all-statute inputs, Table 9)?
- Which statutes are commonly missed or confused (Table 5, Table 6)?
- Is information from particular sections of the document useful?
- Does distillation keep performance while reducing size and compute (Table 7)?
- How do model size, memory and inference cost change across experiments?
- Are the Phase-1 representations useful for LSI (compare with the lexical models, Table 8)?

6 Limitations and next steps

Facts that limit the conclusions (the interpretation is the author's):

- One seed per run; runs differ by a few thousandths of macro-F1, which is within seed noise.
- Stackers are trained on the validation split (5-fold); their scores are out-of-fold, but stacker settings were chosen on the same split.
- The best submissions (STACK7–11) include lexical TF-IDF models that use no Phase-1 weights; STACK6 is the best Phase-1-only combination.
- Eight 512-token chunks cover the whole document for 88% of validation documents.
- Statute names are masked, which removes the lexical shortcut but also the most direct evidence.

To be written by the author — what you would do next

With more time or compute: ...

7 Conclusions

To be written by the author — conclusions

Summarise what you conclude from the experiments, in your own words.

8 Reproducibility

Token cache	built on first use by `lsi/data.py` (`build_cache`)
First-round experiments	`bash run_queue.sh`
One experiment	`python train.py -config configs/w4_a4_lorma_ep5.yaml`
Teacher logits + KD	`python dump_logits.py -run runs/W1_a4_partial_ep8 ... -out TEACHER4_ens`; `python train.py -config configs/y2_kd4_a4_alpha03.yaml`
Single-run zip	`python make_submission.py -run runs/Y2_kd4_a4_alpha03`
Lexical models	`python tfidf_lr.py -name T1_tfidf_word ...`
Stackers	`python stack.py -all -name STACK6_all`; `python stack_gbm.py -name ... -blend ...`
Figures and tables	`python p5/report/make_figures.py`

To be written by the author — repository link

GitHub: `https://github.com/...`

LLM declaration

To be written by the author — LLM declaration (mandatory)

Fill in the course questionnaire. Describe honestly which parts used an LLM assistant (for example the data pipeline, model and training code, figures and this report's methods/results text) and which parts are your own (design decisions, hyperparameters, analysis, conclusions).

References

[1] H. Bihany, S. Patel, A. Modi. LoRMA: Low-rank multiplicative adaptation for LLMs. Findings of ACL 2025.

[2] T. Dettmers et al. QLoRA: Efficient finetuning of quantized LLMs. NeurIPS 2023.

[3] G. Hinton, O. Vinyals, J. Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.

[4] N. Houlsby et al. Parameter-efficient transfer learning for NLP. ICML 2019.

[5] B. Lester, R. Al-Rfou, N. Constant. The power of scale for parameter-efficient prompt tuning. EMNLP 2021.

[6] X. L. Li, P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. ACL 2021.

[7] Q. Zhang et al. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. ICLR 2023.